

EPISODE 9

System Design LLD & HLD

From Code Design to Distributed Architecture

SOLID Principles

Design Patterns (23 GoF)

LLD Machine Coding

HLD Architecture

CAP Theorem

Interview Framework

LOW LEVEL DESIGN — SOLID & DESIGN PATTERNS

LLD: Code-Level System Design

Low Level Design is about writing maintainable, extensible, testable code. It's the difference between code that works once and code that works forever. SOLID principles and Design Patterns are the vocabulary of LLD.

SOLID Principles



SOLID: five principles that make object-oriented code maintainable and extensible

```
// Single Responsibility Principle
// BAD: UserService does too much
class UserService { createUser() {} sendEmail() {} generatePDF() {} }

// GOOD: each class has ONE job
class UserRepository { create(user) {} findById(id) {} }
class EmailService { send(to, subject, body) {} }
class PDFGenerator { generate(data) {} }

// Open/Closed Principle – add behavior without changing existing code
// BAD: modify existing class for each new discount type
// GOOD: extend via strategy
class DiscountStrategy { calculate(price) { return price; } }
class VIPDiscount extends DiscountStrategy {
  calculate(price) { return price * 0.8; } // 20% off
}
class StudentDiscount extends DiscountStrategy {
  calculate(price) { return price * 0.9; } // 10% off
}

// Dependency Inversion – depend on interface, not implementation
// BAD: OrderService depends on MySQLDatabase directly
// GOOD:
class OrderService {
  constructor(private db: DatabaseInterface) {} // inject abstraction
  createOrder(data) { return this.db.save(data); }
}
// Works with MySQL, MongoDB, or MockDB in tests
```

Design Patterns — Overview

CREATIONAL	STRUCTURAL	BEHAVIORAL
Singleton Factory Method Abstract Factory Builder Prototype	Adapter Facade Proxy Decorator Composite	Observer Strategy Command Iterator State Template Method

23 GoF patterns: know Singleton, Factory, Observer, Strategy, Decorator deeply

```
// Observer Pattern — event-driven notification
class EventEmitter {
  #listeners = new Map();
  on(event, cb) { (this.#listeners.get(event) || this.#listeners.set(event, [])).get(event)).push(cb);
  emit(event, data) { this.#listeners.get(event)?.forEach(cb => cb(data)); }
}

// Strategy Pattern — swappable algorithms
class Sorter {
  constructor(strategy) { this.strategy = strategy; }
  sort(data) { return this.strategy.sort(data); }
}
const mergeSort = { sort: (arr) => arr.slice().sort() };
const quickSort = { sort: (arr) => arr.slice().sort((a,b) => a-b) };

// Singleton — one instance only
class Database {
  static #instance = null;
  static getInstance() {
    if (!Database.#instance) Database.#instance = new Database();
    return Database.#instance;
  }
}
```

HIGH LEVEL DESIGN — ARCHITECTURE & SCALE

HLD: System Architecture

HLD Core Infrastructure Components

Load Balancer

Distributes traffic

API Gateway

Auth, rate limit, CORS

Cache (Redis)

Sub-ms read, TTL

Message Queue

Async, decoupled, idempotent

CDN

Edge caching, static content

SQL DB

ACID, joins, normalized

NoSQL DB

Flexible schema, horizontal scale

Search (Elastic)

Full-text, faceted, relevance

Object Storage

S3 — files, images

Monitor +Alert

Logs, metrics, alerts

Know each component's purpose, trade-offs, and when to use it

1. Requirements (5 min)

Functional: what it does

Non-functional: scale, latency

Out of scope: what to skip

2. Estimation (3 min)

Users: DAU, QPS

Storage: GB/TB per year

Bandwidth: data in/out

3. API Design (5 min)

Define endpoints

RESTful contracts

Request/Response

4. High-Level Design (10 min)

Draw components

Clients→CDN→LB

→Services→DB

5. Deep Dive (15 min)

Bottlenecks

Scaling decisions

DB schema/sharding

6. Trade-offs (5 min)

SQL vs NoSQL

Pull vs Push

CAP theorem

System design interview: follow this 6-step framework in every interview

Decision	Option A	Option B	Choose When
SQL vs NoSQL	PostgreSQL	MongoDB	SQL: ACID; NoSQL: scale/flex
Cache strategy	Cache-aside	Write-through	Aside: read-heavy; Through: write-heavy
Sync vs Async	REST API	Message Queue	Sync: realtime; Async: decouple
Push vs Pull	WebSockets	Polling	Push: low latency; Pull: simple ok
Monolith vs Micro	Monolith	Microservices	Mono: <10 devs; Micro: large teams
CP vs AP (CAP)	Strong consistency	High availability	Banks: CP; Social media: AP

LLD Practice Problems

- ✓ Parking Lot: ParkingLot, ParkingFloor, ParkingSpot, Vehicle, Ticket classes
- ✓ Vending Machine: State pattern — Idle/HasMoney/Dispensing/OutOfStock states
- ✓ BookMyShow: Screen, Show, Seat, Booking, Payment with Strategy pattern
- ✓ Splitwise: User, Group, Expense, Split, Balance with Observer for notifications
- ✓ Elevator System: Elevator, Request, Scheduler (SCAN algorithm) pattern

HLD Practice Problems

- ✓ Design URL Shortener: hash function, DB, cache layer, analytics
- ✓ Design Twitter: fan-out on write vs read, timeline generation, CDN
- ✓ Design WhatsApp: WebSockets, message queue, delivery receipts, E2E encryption
- ✓ Design YouTube: video upload pipeline, CDN, adaptive bitrate, search
- ✓ Design Uber: geospatial indexing, driver matching, surge pricing, payments